CONFIDENTIAL ACADEMIC SUBMISSION - CSE499 review and grading only.

Mercury Project

User Manual and Demo Guide

Build, run, test, and demonstrate Mercury RC 1.2.

Course	CSE499 Senior Project
Author	Matthew D. Barker
Version	RC 1.2 (1.2.0-rc.1)
Date	July 2026
Use	Academic review and grading only

Document Purpose

This manual explains how to build Mercury, run the automated tests, start the console and WinForms demonstrations, execute security scenarios, and interpret expected results.

Audience

This manual is written for CSE499 evaluators, reviewers, and developers who need to run and understand Mercury RC 1.2.

Prerequisites

- Visual Studio 2022 or later, or the .NET command-line tools.
- .NET SDK 10 matching the current demo and test targets.
- Windows for Mercury.Demo.WinForms.
- Windows, macOS, or Linux for Mercury.Demo.Console where the .NET SDK is supported.
- Access to the Mercury solution and source projects.
- Permission to bind the configured TCP loopback ports when using TCP scenarios.

Project Layout

Project	Purpose
Mercury.Abstractions	Shared contracts, primitives, envelope interfaces, crypto, replay, transport, logging, and result abstractions.
Mercury.Core	Mercury client, factories, envelope service, codecs, replay implementation, and pipeline coordination.
Mercury.Provider.AesGcm	AES-GCM authenticated-encryption provider.
Mercury.Provider.ChaCha20	ChaCha20-Poly1305 authenticated-encryption provider.
Mercury.Transport.InMemory	In-memory transport used by tests and demonstrations.
Mercury.Transport.Tcp	Framed TCP transport.
Mercury.Transport.FileDrop	File-based transport plugin.
Mercury.Demo.Console	Cross-platform console demonstration.
Mercury.Demo.WinForms	Windows graphical demonstration and controlled attack simulator.
Mercury.Tests	xUnit automated test suite.

Build Instructions

Visual Studio

1. Open the Mercury solution.
2. Allow NuGet restore to complete.
3. Build the solution.
4. Confirm that the build completes without errors.
5. Select Mercury.Demo.Console or Mercury.Demo.WinForms as the startup project for demonstration.

Command Line

```
dotnet restore
dotnet build
```

For a clean release build:

```
dotnet build --configuration Release
```

Run Automated Tests

6. Open Test Explorer in Visual Studio or use the command line.
7. Run all tests in Mercury.Tests.
8. Confirm that no test fails.
9. Confirm that only the two pending AES-CCM tests are skipped.
10. Record the result in the Test Plan and Test Report.

```
dotnet test
dotnet test --configuration Release
```

Latest RC 1.2 verification: 330 tests discovered, 328 passed, 2 AES-CCM tests skipped, and 0 failed.

Run the Console Demo

```
dotnet run --project Mercury.Demo.Console
```

11. Start the console application.
12. Select the available provider, codec, transport, or file scenario from the menu.
13. Enter or accept a payload.
14. Run the exchange.
15. Verify that the recovered payload matches the original after successful validation.
16. Use chunked file scenarios to demonstrate binary payload handling where available.

Run the WinForms Demo

```
dotnet run --project Mercury.Demo.WinForms
```

17. Start Mercury.Demo.WinForms on Windows.
18. Choose a Crypto Provider, Transport, Envelope Codec, chunking option, chunk size, and logging level in the Configuration panel.
19. Select Apply Configuration.
20. Enter a payload in the Send Data panel or choose Send File.
21. Select Send.
22. Review the Recovered Payload / Result panel, event log, SecureEnvelope viewer, and security-status indicators.
23. Use the Security Demonstrations buttons for Replay Attack, Tamper Attack, or Wrong Key Attack when the configured scenario supports them.

Configuration Guide

Setting	Purpose	Notes
Crypto Provider	Selects AES-GCM or ChaCha20-Poly1305.	AES-CCM is pending and is not part of the completed RC 1.2 provider set.

Setting	Purpose	Notes
Transport	Selects the available delivery plugin.	TCP supports the controlled in-transit attack simulator in the WinForms demo.
Envelope Codec	Selects Binary or JSON SecureEnvelope encoding.	Both codecs carry the same authenticated envelope content.
Chunking	Enables chunked transfer for larger data or files.	Chunk size controls the payload portion sent per exchange.
Logging	Controls demo event detail.	Verbose logging is useful for evaluation but must not expose key material.
Apply Configuration	Rebuilds the demo client and dependencies using the selected settings.	Apply changes before sending a new scenario.

Demo Scenario Guide

Secure Round Trip

Purpose: Demonstrates that a payload is protected by the sender, moved as a protected frame, authenticated by the receiver, and released only after validation.

24. Apply a valid provider, codec, and transport configuration.
25. Enter a payload.
26. Select Send.
27. Verify a success result and matching recovered payload.
28. Review the protected frame preview to confirm that plaintext is not transported directly.

Replay Attack

Purpose: Demonstrates that a previously accepted protected frame cannot be accepted again in the same replay-protection context.

29. Send a valid payload so the attack simulator can retain a complete frame.
30. Select Replay Attack.
31. Send the next exchange as directed by the demo.
32. Verify that the replayed frame is rejected and no plaintext is released.
33. Confirm that Replay Protection remains active in the security-status panel.

Tamper Attack

Purpose: Demonstrates that changing a protected payload in transit causes authentication failure.

34. Configure TCP with either Binary or JSON encoding.
35. Select Tamper Attack.
36. Send a payload.
37. Verify that the attack simulator modifies the protected payload without decrypting it.
38. Verify that the receiver blocks the communication and releases no plaintext.
39. Review the Integrity Check and Tamper Detection status indicators.

Wrong Key Attack

Purpose: Demonstrates that a receiver using mismatched symmetric key material cannot authenticate and decrypt the payload.

40. Select Wrong Key Attack.
41. Send a protected payload.
42. Allow the demo to use mismatched receiving key material.
43. Verify AuthenticationFailed or the corresponding controlled demo result.

44. Confirm that no recovered plaintext is displayed.

TCP Transport

Purpose: Demonstrates complete protected-frame exchange over framed TCP.

45. Select TCP and apply the configuration.
46. Confirm that the listener and client endpoints start successfully.
47. Send a protected payload.
48. Verify successful receive or the intended controlled attack result.
49. If the port is unavailable, stop other demo instances and restart the configuration.

Secure File Transfer

Purpose: Demonstrates that Mercury protects arbitrary binary data rather than only text strings.

50. Enable chunking when appropriate and select a chunk size.
51. Select Send File and choose a source file.
52. Run the transfer.
53. Verify successful completion and matching recovered data or file status.
54. Use a writable output location when the host creates a recovered file.

Expected Results

Scenario	Expected Result
Secure round trip	Success result and recovered payload matches the original.
Replay attack	Duplicate protected frame is rejected and no plaintext is released.
Tamper attack	Authentication or integrity check fails and receive is blocked.
Wrong key	Authentication fails and no plaintext is released.
Binary or JSON codec	Valid frames round trip; malformed or wrong-codec frames are rejected.
TCP transport	Complete frame is delivered and processed; oversized frames are rejected.
File transfer	Recovered binary data matches the source after successful validation.

Troubleshooting

Problem	Likely Cause	Action
Build fails due to SDK	Installed SDK does not match the project target.	Install .NET SDK 10 or use the approved course environment.
Test Explorer stops on FormatException	Visual Studio is configured to break when FormatException is thrown even if Mercury catches it.	Clear the thrown-exception checkbox for System.FormatException in Exception Settings.
TCP demo cannot connect	Port conflict, stale demo instance, or startup timing.	Close other demo instances, verify loopback port availability, and reapply the configuration.
Replay is not detected	The replay protector or receiving client was rebuilt between deliveries.	Use the same configured receiving context and do not reset replay state before the second delivery.
Wrong-key scenario succeeds unexpectedly	Both endpoints are using the same key provider configuration.	Reapply the wrong-key scenario so receiving key material is intentionally mismatched.
Tamper result reports malformed frame	The attack modified envelope structure rather than the protected payload, or stale demo binaries are running.	Clean and rebuild the demo; confirm the current TcpAttackSimulator is used.

Problem	Likely Cause	Action
File transfer cannot write output	Output path is invalid or not writable.	Choose a writable folder and verify file permissions.
Unexpected two skipped tests	AES-CCM remains pending.	This is expected for RC 1.2; any additional skip or failure requires investigation.

Safety Notes

- Do not treat demonstration or test keys as production secrets.
- Do not publish project materials without written permission from the copyright owner.
- Do not claim DoD certification, production readiness, classified use, or regulatory approval.
- Do not use Mercury RC 1.2 as a production key-management system.
- Use the demos for academic evaluation and controlled demonstration.